

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY (ICDFA)



SBT-DF203 LAB 2 REPORT

HTTP Analysis Using Wireshark: Embedded Image Traffic
Basic Networking Skills for Digital Forensics

Student Name: Abubakari-Sadique Hamidu
Student ID: 2025-FWSD-11392

Instructor: Instructor: Aminu Idris (CCNA, CompTIA, CEH, OSCP,
CISSP, CISM | Founder, ICDFA)

Case/Lab Identifier: SBT-DF203-Lab2-AbubakariSadiqueHamidu

Date: 10 September 2026

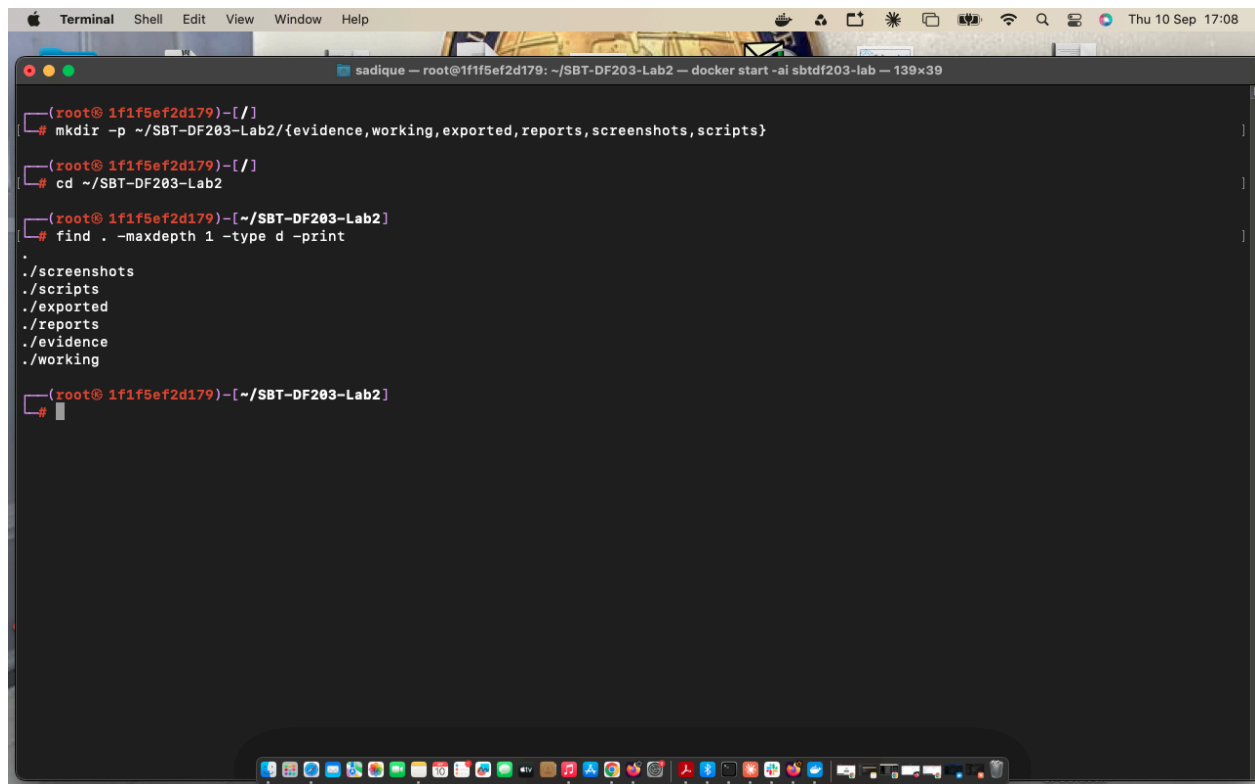
Executive Summary

This lab builds directly on Lab 1. Instead of a webpage with just plain text on it, this time I built a webpage with a picture embedded in it and looked closely at what happens differently when a browser has to fetch two separate things, the page itself and the image instead of just one. I also compared that against curl, a simple command-line tool, to show that curl does not behave like a real browser when it comes to embedded content.

All of this was done inside the same isolated Kali Linux container on my own laptop that I used for Lab 1. Nothing here touched a real website or anyone else's computer. This report is written in the order I actually did the work, in plain words, with technical terms explained the first time they come up.

Step 1: Setting Up a Workspace

I reused the same Kali Linux container from Lab 1, and inside it created a fresh set of folders for this lab: one for the original recording of the network traffic, one for a safe working copy, one for anything I export out of the recording, one for reports, and one for screenshots. Same reasoning as before: keeping the original untouched means I always have something to fall back on if anything goes wrong while I'm working.

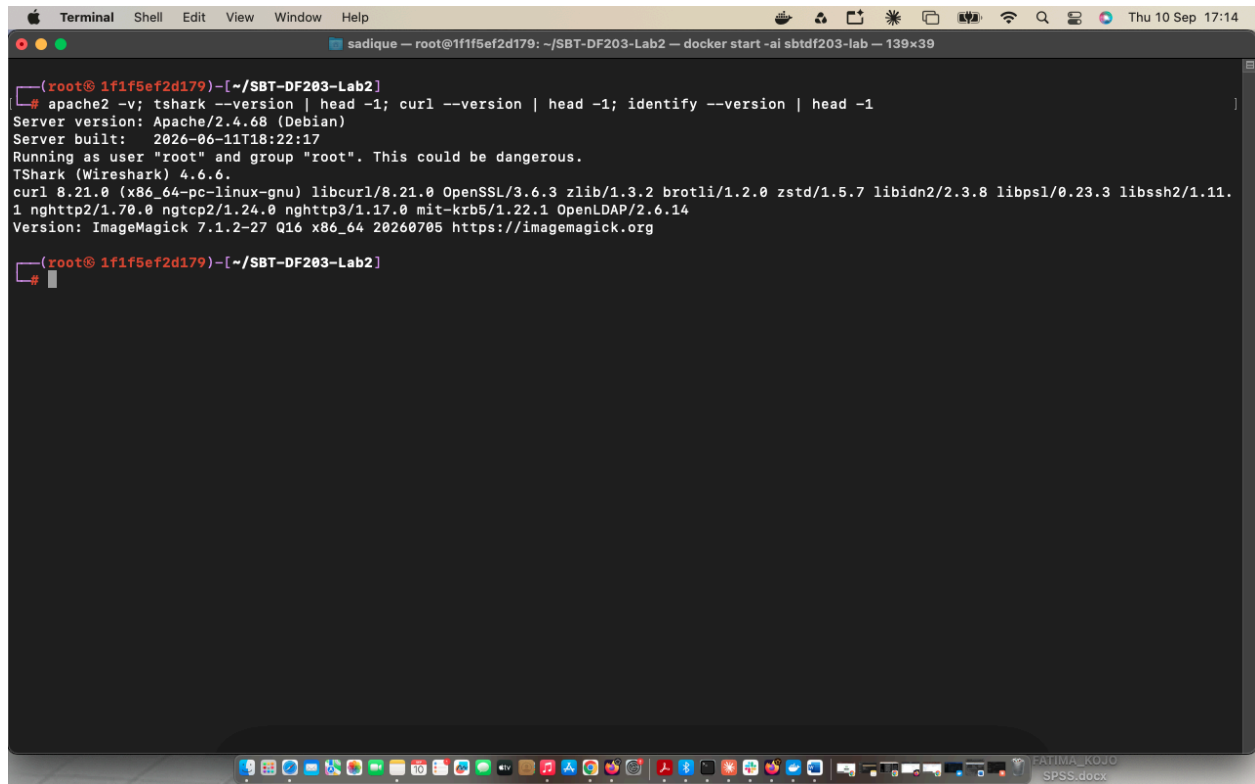
A screenshot of a macOS desktop with a terminal window open. The terminal window has a title bar that reads "Terminal" and a menu bar with "Shell", "Edit", "View", "Window", and "Help". The terminal content shows a user named "sadique" at a root prompt in a Kali Linux container. The user runs a series of commands to create a directory structure. The commands and their outputs are as follows:

```
(root@ 1f1f5ef2d179)-[/]  
# mkdir -p ~/SBT-DF203-Lab2/{evidence,working,exported,reports,screenshots,scripts}  
(root@ 1f1f5ef2d179)-[/]  
# cd ~/SBT-DF203-Lab2  
(root@ 1f1f5ef2d179)-[~/SBT-DF203-Lab2]  
# find . -maxdepth 1 -type d -print  
.  
./screenshots  
./scripts  
./exported  
./reports  
./evidence  
./working  
(root@ 1f1f5ef2d179)-[~/SBT-DF203-Lab2]  
#
```

The terminal window is titled "sadique — root@1f1f5ef2d179: ~/SBT-DF203-Lab2 — docker start -ai sbtdf203-lab — 139x39". The macOS dock is visible at the bottom of the screen.

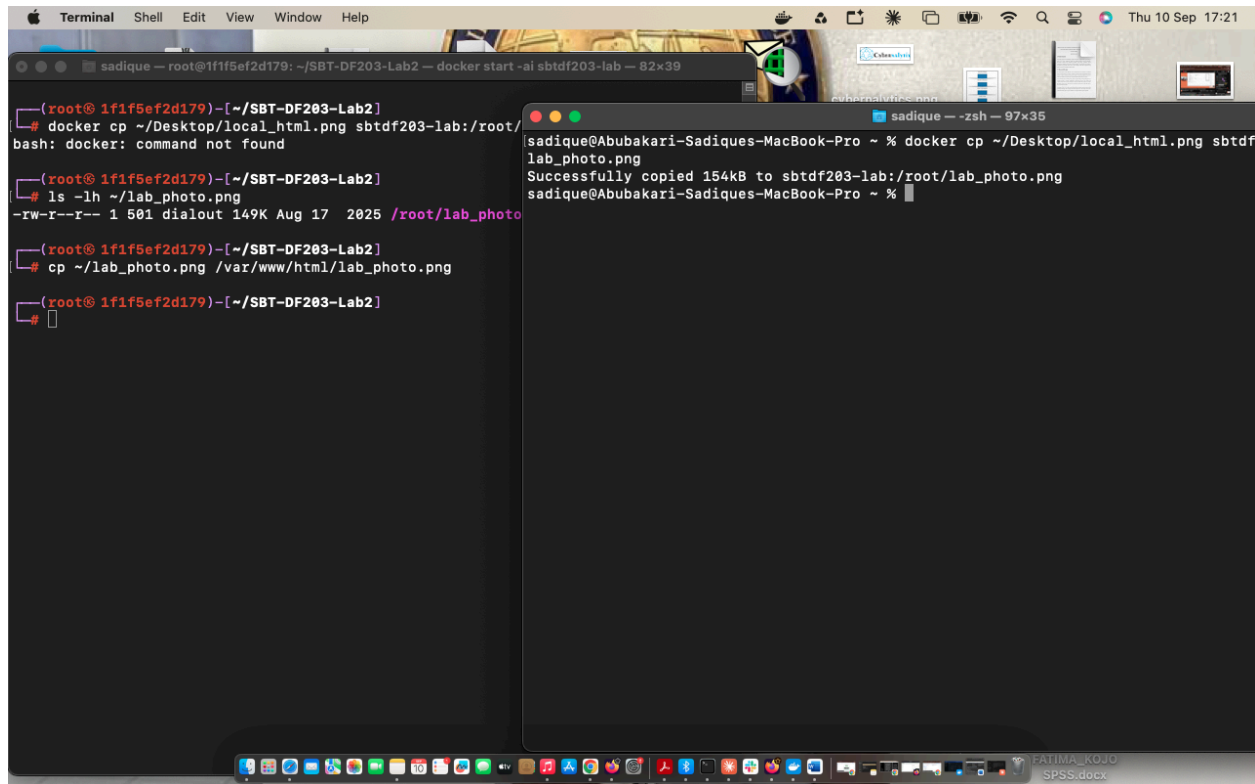
Step 2 Checking the Tools Were Still Ready

Since I was reusing the same container, I didn't need to reinstall anything. Apache, curl, and tshark were already there from Lab 1. I did add one small extra tool this time: "identify", which comes from a program called ImageMagick and can read an image file and tell you its real format and dimensions. That let me confirm my image file was genuinely a valid image, not just a file with a misleading name.

A screenshot of a macOS Terminal window. The title bar shows 'Terminal' and standard window controls. The terminal window title is 'sadique — root@1f1f5ef2d179: ~/SBT-DF203-Lab2 — docker start -ai sbtdf203-lab — 139x39'. The prompt is '(root@1f1f5ef2d179)~[~/SBT-DF203-Lab2]'. The user enters the command '# apache2 -v; tshark --version | head -1; curl --version | head -1; identify --version | head -1'. The output shows: 'Server version: Apache/2.4.68 (Debian)', 'Server built: 2026-06-11T18:22:17', 'Running as user "root" and group "root". This could be dangerous.', 'TShark (Wireshark) 4.6.6.', 'curl 8.21.0 (x86_64-pc-linux-gnu) libcurl/8.21.0 OpenSSL/3.6.3 zlib/1.3.2 brotli/1.2.0 zstd/1.5.7 libidn2/2.3.8 libpsl/0.23.3 libssh2/1.11.1 nghttp2/1.70.0 nghttp/1.24.0 nghttp3/1.17.0 mit-krb5/1.22.1 OpenLDAP/2.6.14', and 'Version: ImageMagick 7.1.2-27 Q16 x86_64 20260705 https://imagemagick.org'. The prompt returns to '(root@1f1f5ef2d179)~[~/SBT-DF203-Lab2]'. The macOS dock is visible at the bottom with various application icons, including a document titled 'FATIMA_KOJO SPSS.docx'.

Step 3: Building the Webpage With an Embedded Image

I copied a photo from my own laptop into the container, renamed it lab_photo.png, and placed it in the same folder Apache serves pages from. Then I built a simple webpage (image.html) with an tag pointing at that photo. This is the one line of HTML that tells a browser "go and fetch this other file too, and show it here." I checked the file with both "file" and "identify" to confirm it was a real, valid PNG image and not corrupted.



Step 4 Fingerprinting the Originals Before Capturing Anything

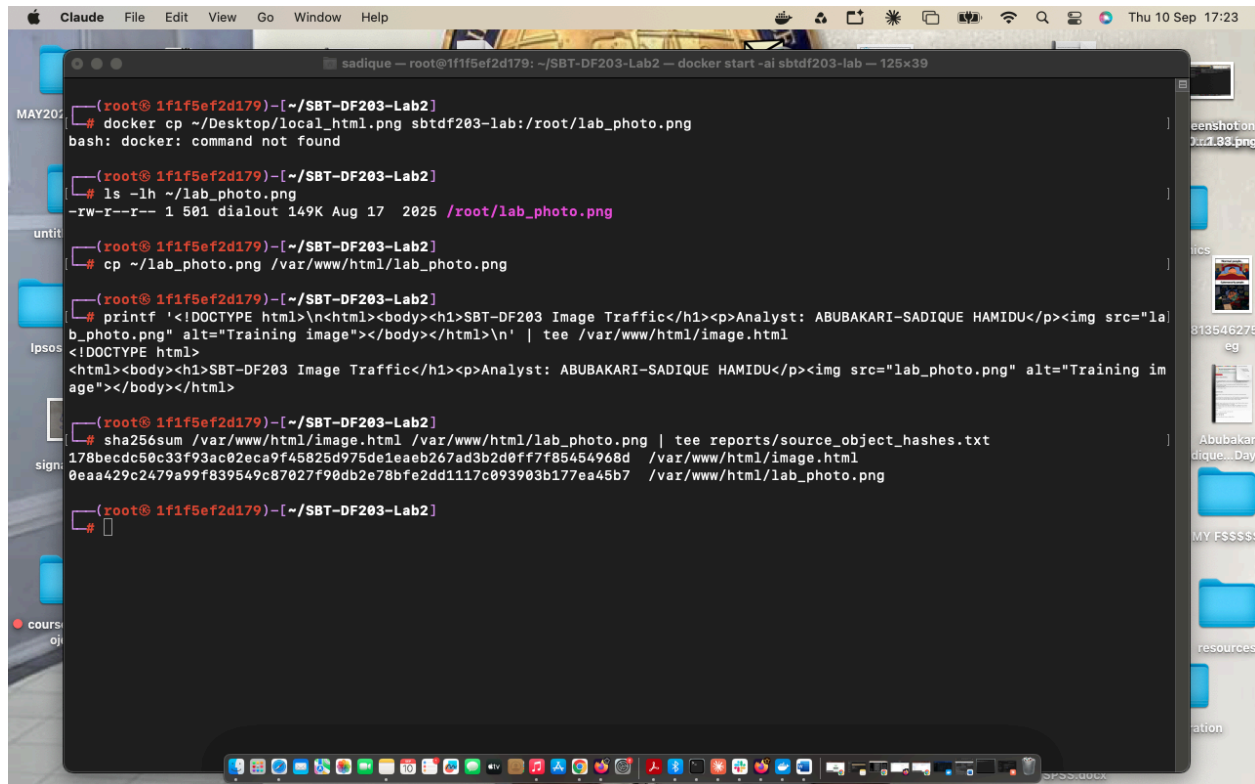
Before recording any traffic, I made a digital fingerprint (a SHA-256 hash) of both original files sitting on the server: image.html and lab_photo.png. The point of doing this now, before anything else happens, is so that later on I can prove whatever gets sent over the network and whatever I pull back out of the recording is exactly the same as what started on the server, byte for byte.

Source file fingerprint image.html:

```
178becdc50c33f93ac02eca9f45825d975de1eae267ad3b2d0ff7f85454968d
```

Source file fingerprint lab_photo.png:

```
0eaa429c2479a99f839549c87027f90db2e78bfe2dd1117c093903b177ea45b7
```



```
(root@ 1f1f5ef2d179)~[/SBT-DF203-Lab2]
# docker cp ~/Desktop/local_html.png sbtdf203-lab:/root/lab_photo.png
bash: docker: command not found

(root@ 1f1f5ef2d179)~[/SBT-DF203-Lab2]
# ls -lh ~/lab_photo.png
-rw-r--r-- 1 501 dialout 149K Aug 17 2025 /root/lab_photo.png

(root@ 1f1f5ef2d179)~[/SBT-DF203-Lab2]
# cp ~/lab_photo.png /var/www/html/lab_photo.png

(root@ 1f1f5ef2d179)~[/SBT-DF203-Lab2]
# printf '<!DOCTYPE html>\n<html><body><h1>SBT-DF203 Image Traffic</h1><p>Analyst: ABUBAKARI-SADIQUE HAMIDU</p></body></html>\n' | tee /var/www/html/image.html
<!DOCTYPE html>
<html><body><h1>SBT-DF203 Image Traffic</h1><p>Analyst: ABUBAKARI-SADIQUE HAMIDU</p></body></html>

(root@ 1f1f5ef2d179)~[/SBT-DF203-Lab2]
# sha256sum /var/www/html/image.html /var/www/html/lab_photo.png | tee reports/source_object_hashes.txt
178becdc50c33f93ac02eca9f45825d975de1eae267ad3b2d0ff7f85454968d /var/www/html/image.html
0eaa429c2479a99f839549c87027f90db2e78bfe2dd1117c093903b177ea45b7 /var/www/html/lab_photo.png

(root@ 1f1f5ef2d179)~[/SBT-DF203-Lab2]
#
```

Step 5: Viewing the Page in a Real Browser

For Lab 1, I used curl to fetch the page. This time, the point of the lab was to see how a real browser behaves, so I opened the page in Safari on my own Mac instead. Docker was set up to forward a port on my Mac (8080) through to port 80 inside the container, so visiting "localhost:8080" in Safari reached the webpage running inside the container. The page loaded properly, showing the embedded photo alongside the page content.

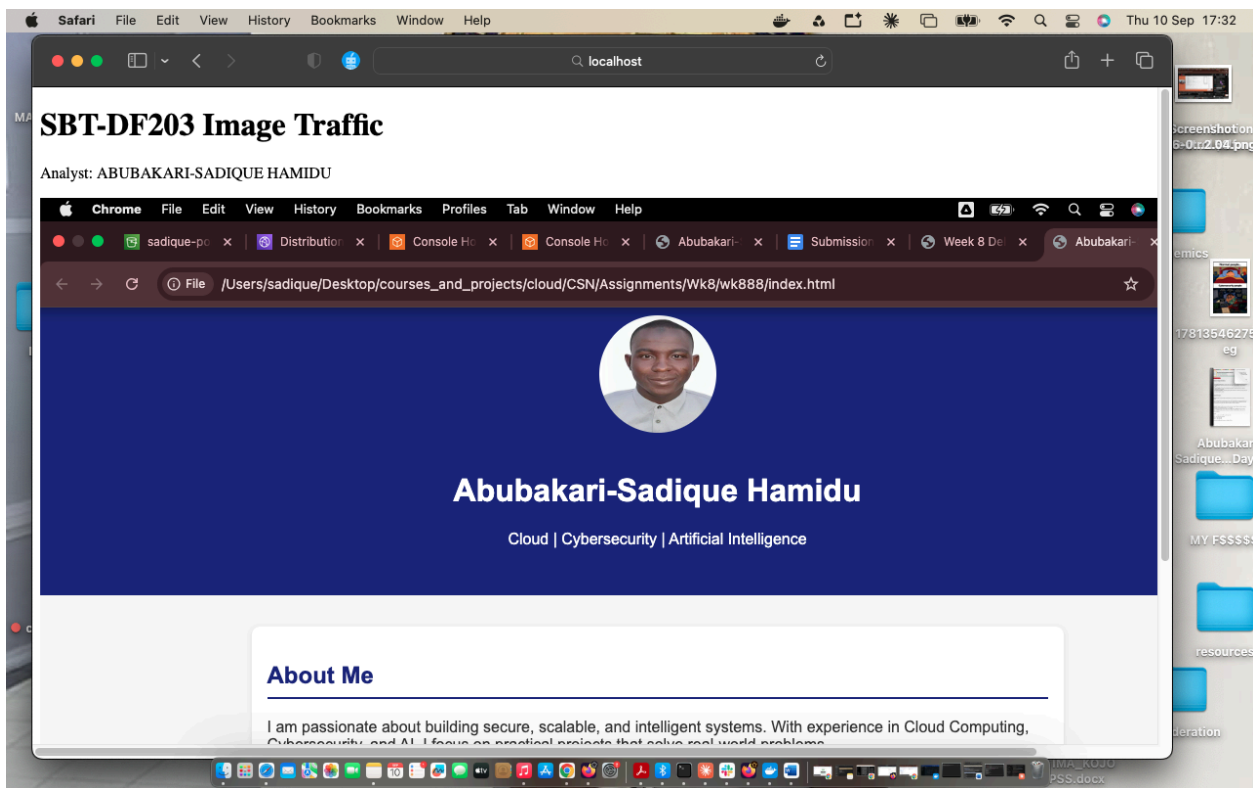
```
Terminal Shell Edit View Window Help
sadique — root@1f1f5ef2d179: ~/SBT-DF203-Lab2 — docker start -ai sbtdf203-lab — 125x39

(root@1f1f5ef2d179)-[~/SBT-DF203-Lab2]
# service apache2 start
Starting Apache httpd web server: apache2AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 172.17.0.2. Set the 'ServerName' directive globally to suppress this message

(root@1f1f5ef2d179)-[~/SBT-DF203-Lab2]
# file /var/www/html/lab_photo.png; identify /var/www/html/lab_photo.png; ls -lh /var/www/html/image.html /var/www/html/lab_photo.png; curl -I http://127.0.0.1/image.html; curl -I http://127.0.0.1/lab_photo.png
/var/www/html/lab_photo.png: PNG image data, 1280 x 800, 8-bit/color RGBA, non-interlaced
/var/www/html/lab_photo.png PNG 1280x800 1280x800+0+0 8-bit sRGB 151792B 0.000u 0:00.000
-rw-r--r-- 1 root root 161 Sep 10 17:22 /var/www/html/image.html
-rw-r--r-- 1 root root 149K Sep 10 17:20 /var/www/html/lab_photo.png
HTTP/1.1 200 OK
Date: Thu, 10 Sep 2026 17:29:07 GMT
Server: Apache/2.4.68 (Debian)
Last-Modified: Thu, 10 Sep 2026 17:22:04 GMT
ETag: "a1-65b2434e07e37"
Accept-Ranges: bytes
Content-Length: 161
Vary: Accept-Encoding
Content-Type: text/html

HTTP/1.1 200 OK
Date: Thu, 10 Sep 2026 17:29:07 GMT
Server: Apache/2.4.68 (Debian)
Last-Modified: Thu, 10 Sep 2026 17:20:20 GMT
ETag: "250f0-65b242eb6bbf2"
Accept-Ranges: bytes
Content-Length: 151792
Content-Type: image/png

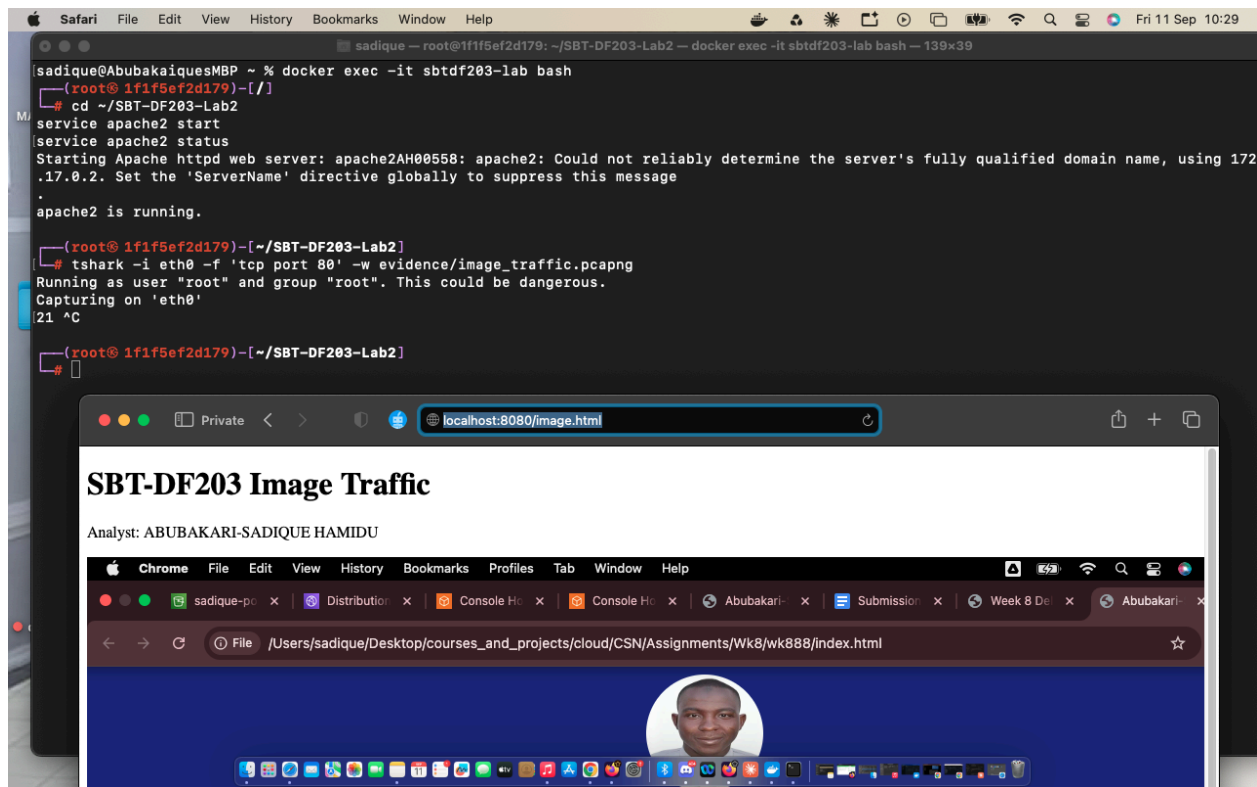
(root@1f1f5ef2d179)-[~/SBT-DF203-Lab2]
```



Step 6: Recording the Browser's Traffic

This is where something genuinely useful came up. My first instinct was to record traffic on "lo," the loopback interface, meaning traffic a computer sends to itself, since that's what worked in Lab 1. But this time, nothing showed up. The reason is that Safari was running on my Mac, not inside the container, and reaching the webpage through Docker's port-forwarding (8080 → 80). That connection comes in from outside the container, through Docker's own internal network bridge, so it actually shows up on the container's "eth0" interface (its regular network connection), not on loopback.

Once I switched the recording to eth0 and reopened the page in Safari, it worked properly: the recording captured 21 packets in total.



Step 7 Keeping the Evidence Safe

Same as Lab 1: I made a working copy of the recording straight away, and left the original file untouched. Fingerprinting both files afterward confirmed they were identical.

Fingerprint (same for both files):

```
b5616b94840cf82c7861d41b6182d9e7bbec89f9458ca362fe5ec6b5e667f757
```



```
Terminal Shell Edit View Window Help
sadique — root@1f1f5ef2d179: ~/SBT-DF203-Lab2 — docker exec -it sbtdf203-lab bash — 139x39

sadique@AbubakaiquesMBP ~ % docker exec -it sbtdf203-lab bash
(root@1f1f5ef2d179)-[/]
# cd ~/SBT-DF203-Lab2
service apache2 start
service apache2 status
Starting Apache httpd web server: apache2AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 172
.17.0.2. Set the 'ServerName' directive globally to suppress this message
.
apache2 is running.

(root@1f1f5ef2d179)-[/SBT-DF203-Lab2]
# tshark -i eth0 -f 'tcp port 80' -w evidence/image_traffic.pcapng
Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'
^C

(root@1f1f5ef2d179)-[/SBT-DF203-Lab2]
# cp --preserve=timestamps evidence/image_traffic.pcapng working/image_traffic_working.pcapng
sha256sum evidence/image_traffic_working.pcapng | tee reports/capture_hashes.txt
b5616b94840cf82c7861d41b6182d9e7bbec89f9458ca362fe5ec6b5e667f757 evidence/image_traffic.pcapng
b5616b94840cf82c7861d41b6182d9e7bbec89f9458ca362fe5ec6b5e667f757 working/image_traffic_working.pcapng

(root@1f1f5ef2d179)-[/SBT-DF203-Lab2]
#
```

```
Terminal Shell Edit View Window Help
sadique — root@1f1f5ef2d179: ~/SBT-DF203-Lab2 — docker exec -it sbtdf203-lab bash — 139x39

(root@1f1f5ef2d179)-[/SBT-DF203-Lab2]
# tshark -r working/image_traffic_working.pcapng
Running as user "root" and group "root". This could be dangerous.
1 0.000000000 192.168.65.1 → 172.17.0.2 TCP 74 49426 → 80 [SYN] Seq=0 Win=65408 Len=0 MSS=65495 TSval=3814094316 TSecr=0 WS=128
2 0.004215621 172.17.0.2 → 192.168.65.1 TCP 74 80 → 49426 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 TSval=1570038762 TSecr=381
4094316 WS=128
3 0.062892569 192.168.65.1 → 172.17.0.2 TCP 66 49426 → 80 [ACK] Seq=1 Ack=1 Win=65408 Len=0 TSval=3814094348 TSecr=1570038762
4 0.204406020 192.168.65.1 → 172.17.0.2 HTTP 513 GET /image.html HTTP/1.1
5 0.204453401 172.17.0.2 → 192.168.65.1 TCP 66 80 → 49426 [ACK] Seq=1 Ack=448 Win=65152 Len=0 TSval=1570038963 TSecr=3814094521
6 0.209926414 172.17.0.2 → 192.168.65.1 HTTP 565 HTTP/1.1 200 OK (text/html)
7 0.210547851 192.168.65.1 → 172.17.0.2 TCP 66 49426 → 80 [ACK] Seq=448 Ack=500 Win=523776 Len=0 TSval=3814094527 TSecr=1570038968
8 0.296238327 192.168.65.1 → 172.17.0.2 HTTP 552 GET /lab_photo.png HTTP/1.1
9 0.300729155 172.17.0.2 → 192.168.65.1 TCP 65549 HTTP/1.1 200 OK
10 0.301667753 172.17.0.2 → 192.168.65.1 TCP 65549 80 → 49426 [ACK] Seq=65983 Ack=934 Win=65536 Len=65483 TSval=1570039060 TSecr=38140
94613
11 0.303668403 172.17.0.2 → 192.168.65.1 HTTP 21181 HTTP/1.1 200 OK (PNG)
12 0.303916922 172.17.0.2 → 192.168.65.1 TCP 65534 [TCP Retransmission] 80 → 49426 [ACK] Seq=500 Ack=934 Win=65536 Len=65468 TSval=157
0039062 TSecr=3814094613
13 0.303946819 172.17.0.2 → 192.168.65.1 TCP 81 [TCP Retransmission] 80 → 49426 [PSH, ACK] Seq=65968 Ack=934 Win=65536 Len=15 TSval=15
70039062 TSecr=3814094613 [TCP PDU reassembled in 11]
14 0.303958163 172.17.0.2 → 192.168.65.1 TCP 65534 [TCP Retransmission] 80 → 49426 [ACK] Seq=65983 Ack=934 Win=65536 Len=65468 TSval=1
570039062 TSecr=3814094613 [TCP PDU reassembled in 11]
15 0.304215976 172.17.0.2 → 192.168.65.1 TCP 81 [TCP Retransmission] 80 → 49426 [PSH, ACK] Seq=131451 Ack=934 Win=65536 Len=15 TSval=1
570039062 TSecr=3814094613 [TCP PDU reassembled in 11]
16 0.308276099 192.168.65.1 → 172.17.0.2 TCP 66 [TCP Dup ACK 7#1] 49426 → 80 [ACK] Seq=934 Ack=500 Win=524288 Len=0 TSval=3814094622 T
Secr=1570038968
17 0.308281031 192.168.65.1 → 172.17.0.2 TCP 66 49426 → 80 [ACK] Seq=934 Ack=152581 Win=372096 Len=0 TSval=3814094623 TSecr=1570039062
18 5.306919747 172.17.0.2 → 192.168.65.1 TCP 66 80 → 49426 [FIN, ACK] Seq=152581 Ack=934 Win=65536 Len=0 TSval=1570044064 TSecr=381409
4623
19 5.307519769 192.168.65.1 → 172.17.0.2 TCP 66 49426 → 80 [ACK] Seq=934 Ack=152582 Win=523904 Len=0 TSval=3814099623 TSecr=1570039062
20 5.307987774 192.168.65.1 → 172.17.0.2 TCP 66 49426 → 80 [FIN, ACK] Seq=934 Ack=152582 Win=524288 Len=0 TSval=3814099624 TSecr=15700
44064
21 5.308017008 172.17.0.2 → 192.168.65.1 TCP 66 80 → 49426 [ACK] Seq=152582 Ack=935 Win=65536 Len=0 TSval=1570044065 TSecr=3814099624

(root@1f1f5ef2d179)-[/SBT-DF203-Lab2]
#
```


Step 8 Proving the Browser Made Two Separate Requests

The whole point of this lab is embedded content, so the key thing to prove is that the browser didn't just ask for one thing it asked for two: the HTML page itself, and then the image referenced inside it. Pulling just the requests out of the recording showed exactly that:

Frame	Time	Request
4	10:28:51.100088	GET /image.html
8	10:28:51.191913	GET /lab_photo.png

And the two replies that came back:

Frame	Time	Status	Type	Size
6	10:28:51.105609	200 OK	text/html	163 bytes
11	10:28:51.199351	200 OK	image/png	151,792 bytes

In plain terms: the browser read the HTML first, found the `<img>` tag inside it, and only then went back and asked for the photo separately, about 0.09 seconds later. A page with an embedded image is never just one request; it's always at least two, because the image is a completely separate file that has to be fetched on its own.

```
Claude File Edit View Go Window Help
sadique -- root@1f1f5ef2d179: ~/SBT-DF203-Lab2 -- docker exec -it sbtdf203-lab bash -- 139x39

(root@1f1f5ef2d179)~/SBT-DF203-Lab2
# tshark -r working/image_traffic_working.pcapng -Y 'http.request' -T fields -e frame.number -e frame.time -e tcp.stream -e ip.src -e tcp
.srcport -e ip.dst -e tcp.dstport -e http.request.method -e http.request.uri -e http.host | tee reports/http_object_requests.tsv
Running as user "root" and group "root". This could be dangerous.
4      2026-09-11T10:28:51.100088387+0000      0      192.168.65.1      49426      172.17.0.2      80      GET      /image.html      localhost:8
080
8      2026-09-11T10:28:51.191912694+0000      0      192.168.65.1      49426      172.17.0.2      80      GET      /lab_photo.png      localhost:8
080

(root@1f1f5ef2d179)~/SBT-DF203-Lab2
#
```

```
Terminal Shell Edit View Window Help
sadique -- root@1f1f5ef2d179: ~/SBT-DF203-Lab2 -- docker exec -it sbtdf203-lab bash -- 139x39

(root@1f1f5ef2d179)~/SBT-DF203-Lab2
# tshark -r working/image_traffic_working.pcapng -Y 'http.request' -T fields -e frame.number -e frame.time -e tcp.stream -e ip.src -e tcp
.srcport -e ip.dst -e tcp.dstport -e http.request.method -e http.request.uri -e http.host | tee reports/http_object_requests.tsv
Running as user "root" and group "root". This could be dangerous.
4      2026-09-11T10:28:51.100088387+0000      0      192.168.65.1      49426      172.17.0.2      80      GET      /image.html      localhost:8
080
8      2026-09-11T10:28:51.191912694+0000      0      192.168.65.1      49426      172.17.0.2      80      GET      /lab_photo.png      localhost:8
080

(root@1f1f5ef2d179)~/SBT-DF203-Lab2
# tshark -r working/image_traffic_working.pcapng -Y 'http.response' -T fields -e frame.number -e frame.time -e tcp.stream -e http.respons
e.code -e http.content_type -e http.content_length | tee reports/http_object_responses.tsv
Running as user "root" and group "root". This could be dangerous.
6      2026-09-11T10:28:51.105608781+0000      0      200      text/html      163
11     2026-09-11T10:28:51.199350770+0000      0      200      image/png      151792

(root@1f1f5ef2d179)~/SBT-DF203-Lab2
#
```

Step 9 How the Connection Was Used

Both requests and both replies travelled over the same single connection between my Mac and the container there was no need to open a second connection just because a second file was requested. Here's the overall summary of that one connection:

192.168.65.1:49426 <-> 172.17.0.2:80
Frames: 21 total (285 kB) Duration: 5.308 seconds

```
Terminal Shell Edit View Window Help
sadique ~ root@1f1f5ef2d179: ~/SBT-DF203-Lab2 — docker exec -it sbtfd203-lab bash — 139x39
```

```
(root@ 1f1f5ef2d179)~/~/SBT-DF203-Lab2]
# tshark -r working/image_traffic_working.pcapng -Y 'tcp.stream==0 && tcp.len>0' -T fields -e frame.number -e frame.time -e ip.len -e tcp[
Running as user "root" and group "root". This could be dangerous.
```

	frame.number	frame.time	ip.len	tcp.len
4	2026-09-11T10:28:51.100088387+0000	499	32	447 1 1
6	2026-09-11T10:28:51.105608781+0000	551	32	499 1 448
8	2026-09-11T10:28:51.191912694+0000	538	32	486 448 500
9	2026-09-11T10:28:51.196411522+0000	65535	32	65483 500 934
10	2026-09-11T10:28:51.197350120+0000	65535	32	65483 65983 934
11	2026-09-11T10:28:51.199350770+0000	21167	32	21115 131466 934
12	2026-09-11T10:28:51.199599280+0000	65520	32	65468 500 934
13	2026-09-11T10:28:51.199629186+0000	67	32	15 65968 934
14	2026-09-11T10:28:51.199640530+0000	65520	32	65468 65983 934
15	2026-09-11T10:28:51.199898343+0000	67	32	15 131451 934

```
(root@ 1f1f5ef2d179)~/~/SBT-DF203-Lab2]
# tshark -r working/image_traffic_working.pcapng -q -z conv,tcp | tee reports/tcp_conversations.txt
Running as user "root" and group "root". This could be dangerous.
```

```
=====
TCP Conversations
Filter:<No Filter>
```

n	<-	->	Total	Relative	Duratio		
	Frames Bytes	Frames Bytes	Frames Bytes	Start			
192.168.65.1:49426	<->	172.17.0.2:80	12 284 kB	9 1535 bytes	21 285 kB	0.00000000	5.
3080							

```
=====
(root@ 1f1f5ef2d179)~/~/SBT-DF203-Lab2]
```

Step 10: How the Image Was Put Back Together

A photo is a lot bigger than a short piece of text, so it doesn't fit inside one single piece of network traffic. It has to be broken into several smaller pieces and reassembled at the other end, a bit like a large parcel being split into several boxes and then unpacked together on arrival.

Wireshark keeps track of this automatically. Looking at the final piece of the image reply, it reported that the full image (151,792 bytes, matching the Content-Length the server declared) was built back up out of 3 separate TCP segments, totalling 152,081 raw bytes once you include a little bit of header information carried alongside the very first piece.

```
Claude File Edit View Go Window Help
sadique -- root@1f1f5ef2d179: ~/SBT-DF203-Lab2 -- docker exec -it sbtdf203-lab bash -- 139x39

(root@1f1f5ef2d179)~/SBT-DF203-Lab2
# tshark -r working/image_traffic_working.pcapng -Y 'tcp.stream==0 && tcp.len>0' -T fields -e frame.number -e frame.time -e ip.len -e tcp
.hdr_len -e tcp.len -e tcp.seq -e tcp.ack | tee reports/image_stream_segments.tsv
Running as user "root" and group "root". This could be dangerous.
4      2026-09-11T10:28:51.100088387+0000      499      32      447      1      1
6      2026-09-11T10:28:51.105608781+0000      551      32      499      1      448
8      2026-09-11T10:28:51.191912694+0000      538      32      486      448      500
9      2026-09-11T10:28:51.196411522+0000      65535      32      65483      500      934
10     2026-09-11T10:28:51.197350120+0000      65535      32      65483      65983      934
11     2026-09-11T10:28:51.199350770+0000      21167      32      21115      131466      934
12     2026-09-11T10:28:51.199599289+0000      65520      32      65468      500      934
13     2026-09-11T10:28:51.199629186+0000      67      32      15      65968      934
14     2026-09-11T10:28:51.199640530+0000      65520      32      65468      65983      934
15     2026-09-11T10:28:51.199898343+0000      67      32      15      131451      934

(root@1f1f5ef2d179)~/SBT-DF203-Lab2
#
```

Step 11: Recovering the Image and the Page From the Recording

Wireshark (and its command-line version, tshark) can pull whole files back out of a recording, not just show you that they were transferred. I used tshark's export-objects feature to do exactly that, and it extracted both files cleanly: image.html and lab_photo.png. I then opened the recovered lab_photo.png file to confirm it displayed properly as a normal photo it did, with no corruption or missing data.

```
Terminal Shell Edit View Window Help
sadique -- root@1f1f5ef2d179: ~/SBT-DF203-Lab2 -- docker exec -it sbtdf203-lab bash -- 139x39

(root@1f1f5ef2d179)-[~/SBT-DF203-Lab2]
# mkdir -p exported/http_objects

(root@1f1f5ef2d179)-[~/SBT-DF203-Lab2]
# tshark -r working/image_traffic_working.pcapng --export-objects http,exported/http_objects
Running as user "root" and group "root". This could be dangerous.
1 0.000000000 192.168.65.1 -> 172.17.0.2 TCP 74 49426 -> 80 [SYN] Seq=0 Win=65408 Len=0 MSS=65495 TSval=3814094316 TSecr=0 WS=128
2 0.004215621 172.17.0.2 -> 192.168.65.1 TCP 74 80 -> 49426 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 TSval=1570038762 TSecr=3814094316 WS=128
3 0.062892569 192.168.65.1 -> 172.17.0.2 TCP 66 49426 -> 80 [ACK] Seq=1 Ack=1 Win=65408 Len=0 TSval=3814094348 TSecr=1570038762
4 0.204406020 192.168.65.1 -> 172.17.0.2 HTTP 513 GET /image.html HTTP/1.1
5 0.204453401 172.17.0.2 -> 192.168.65.1 TCP 66 80 -> 49426 [ACK] Seq=1 Ack=448 Win=65152 Len=0 TSval=1570038963 TSecr=3814094521
6 0.209926414 172.17.0.2 -> 192.168.65.1 HTTP 565 HTTP/1.1 200 OK (text/html)
7 0.210547851 192.168.65.1 -> 172.17.0.2 TCP 66 49426 -> 80 [ACK] Seq=448 Ack=500 Win=523776 Len=0 TSval=3814094527 TSecr=1570038968
8 0.296238327 192.168.65.1 -> 172.17.0.2 HTTP 552 GET /lab_photo.png HTTP/1.1
9 0.300729155 172.17.0.2 -> 192.168.65.1 TCP 65549 HTTP/1.1 200 OK
10 0.301667753 172.17.0.2 -> 192.168.65.1 TCP 65549 80 -> 49426 [ACK] Seq=65983 Ack=934 Win=65536 Len=65483 TSval=1570039060 TSecr=3814094613
11 0.303668403 172.17.0.2 -> 192.168.65.1 HTTP 21181 HTTP/1.1 200 OK (PNG)
12 0.303916922 172.17.0.2 -> 192.168.65.1 TCP 65534 [TCP Retransmission] 80 -> 49426 [ACK] Seq=500 Ack=934 Win=65536 Len=65468 TSval=1570039062 TSecr=3814094613
13 0.303946819 172.17.0.2 -> 192.168.65.1 TCP 81 [TCP Retransmission] 80 -> 49426 [PSH, ACK] Seq=65968 Ack=934 Win=65536 Len=15 TSval=1570039062 TSecr=3814094613 [TCP PDU reassembled in 11]
14 0.303958163 172.17.0.2 -> 192.168.65.1 TCP 65534 [TCP Retransmission] 80 -> 49426 [ACK] Seq=65983 Ack=934 Win=65536 Len=65468 TSval=1570039062 TSecr=3814094613 [TCP PDU reassembled in 11]
15 0.304215976 172.17.0.2 -> 192.168.65.1 TCP 81 [TCP Retransmission] 80 -> 49426 [PSH, ACK] Seq=131451 Ack=934 Win=65536 Len=15 TSval=1570039062 TSecr=3814094613 [TCP PDU reassembled in 11]
16 0.308276099 192.168.65.1 -> 172.17.0.2 TCP 66 [TCP Dup ACK 7#1] 49426 -> 80 [ACK] Seq=934 Ack=500 Win=524288 Len=0 TSval=3814094622 TSecr=1570038968
17 0.308281031 192.168.65.1 -> 172.17.0.2 TCP 66 49426 -> 80 [ACK] Seq=934 Ack=152581 Win=372096 Len=0 TSval=3814094623 TSecr=1570039062
18 5.306919747 172.17.0.2 -> 192.168.65.1 TCP 66 80 -> 49426 [FIN, ACK] Seq=152581 Ack=934 Win=65536 Len=0 TSval=1570044064 TSecr=3814094623
19 5.307519769 192.168.65.1 -> 172.17.0.2 TCP 66 49426 -> 80 [ACK] Seq=934 Ack=152582 Win=523904 Len=0 TSval=3814099623 TSecr=1570039062
20 5.307987774 192.168.65.1 -> 172.17.0.2 TCP 66 49426 -> 80 [FIN, ACK] Seq=934 Ack=152582 Win=524288 Len=0 TSval=3814099624 TSecr=1570044064
21 5.308017008 172.17.0.2 -> 192.168.65.1 TCP 66 80 -> 49426 [ACK] Seq=152582 Ack=935 Win=65536 Len=0 TSval=1570044065 TSecr=3814099624

(root@1f1f5ef2d179)-[~/SBT-DF203-Lab2]
```

Step 12 Proving the Recovered Files Are Genuine

This is the step that ties everything together. I fingerprinted the two files I had just recovered from the network recording, and compared them against the fingerprints I took of the originals back in Step 4, before any of this started.

File	Source fingerprint (before capture)	Recovered fingerprint (after capture)	Match?
image.html	178becdc...54968d	178becdc...54968d	Yes identical
lab_photo.png	0eaa429c...ea45b7	0eaa429c...ea45b7	Yes identical

Both fingerprints matched exactly, both times. That's solid proof that what I pulled back out of the network recording after it had been broken into pieces, sent across a connection, and reassembled is byte-for-byte identical to what left the server in the first place. Nothing was corrupted, dropped, or altered anywhere along the way.

```
Terminal Shell Edit View Window Help
sadique ~ root@1f1f5ef2d179: ~/SBT-DF203-Lab2 — docker exec -it sbtdf203-lab bash — 139x39

10 0.301667753 172.17.0.2 → 192.168.65.1 TCP 65549 80 → 49426 [ACK] Seq=65983 Ack=934 Win=65536 Len=65483 TSval=1570039060 TSecr=38140
94613
11 0.303668403 172.17.0.2 → 192.168.65.1 HTTP 21181 HTTP/1.1 200 OK (PNG)
12 0.303916922 172.17.0.2 → 192.168.65.1 TCP 65534 [TCP Retransmission] 80 → 49426 [ACK] Seq=500 Ack=934 Win=65536 Len=65468 TSval=157
0039062 TSecr=3814094613
13 0.303946819 172.17.0.2 → 192.168.65.1 TCP 81 [TCP Retransmission] 80 → 49426 [PSH, ACK] Seq=65968 Ack=934 Win=65536 Len=15 TSval=15
70039062 TSecr=3814094613 [TCP PDU reassembled in 11]
14 0.303958163 172.17.0.2 → 192.168.65.1 TCP 65534 [TCP Retransmission] 80 → 49426 [ACK] Seq=65983 Ack=934 Win=65536 Len=65468 TSval=1
570039062 TSecr=3814094613 [TCP PDU reassembled in 11]
15 0.304215976 172.17.0.2 → 192.168.65.1 TCP 81 [TCP Retransmission] 80 → 49426 [PSH, ACK] Seq=131451 Ack=934 Win=65536 Len=15 TSval=1
570039062 TSecr=3814094613 [TCP PDU reassembled in 11]
16 0.308276099 192.168.65.1 → 172.17.0.2 TCP 66 [TCP Dup ACK 7#1] 49426 → 80 [ACK] Seq=934 Ack=500 Win=524288 Len=0 TSval=3814094622 T
Secr=1570038968
17 0.308281031 192.168.65.1 → 172.17.0.2 TCP 66 49426 → 80 [ACK] Seq=934 Ack=152581 Win=372096 Len=0 TSval=3814094623 TSecr=1570039062
18 5.306919747 172.17.0.2 → 192.168.65.1 TCP 66 80 → 49426 [FIN, ACK] Seq=152581 Ack=934 Win=65536 Len=0 TSval=1570044064 TSecr=381409
4623
19 5.307519769 192.168.65.1 → 172.17.0.2 TCP 66 49426 → 80 [ACK] Seq=934 Ack=152582 Win=523904 Len=0 TSval=3814099623 TSecr=1570039062
20 5.307987774 192.168.65.1 → 172.17.0.2 TCP 66 49426 → 80 [FIN, ACK] Seq=934 Ack=152582 Win=524288 Len=0 TSval=3814099624 TSecr=15700
44064
21 5.308017008 172.17.0.2 → 192.168.65.1 TCP 66 80 → 49426 [ACK] Seq=152582 Ack=935 Win=65536 Len=0 TSval=1570044065 TSecr=3814099624

(root@1f1f5ef2d179)~/SBT-DF203-Lab2
# find exported/http_objects -maxdepth 1 -type f -ls
200947 152 -rw-r--r-- 1 root root 151792 Sep 11 11:14 exported/http_objects/lab_photo.png
200946 4 -rw-r--r-- 1 root root 161 Sep 11 11:14 exported/http_objects/image.html

(root@1f1f5ef2d179)~/SBT-DF203-Lab2
# file exported/http_objects/*
exported/http_objects/image.html: HTML document, ASCII text
exported/http_objects/lab_photo.png: PNG image data, 1280 x 800, 8-bit/color RGBA, non-interlaced

(root@1f1f5ef2d179)~/SBT-DF203-Lab2
# sha256sum /var/www/html/lab_photo.png exported/http_objects/* | tee reports/extracted_object_hashes.txt
0eaa429c2479a99f839549c87027f90db2e78bfe2dd1117c093903b177ea45b7 /var/www/html/lab_photo.png
178becdc50c33f93ac02eca9f45825d975de1eae267ad3b2d0ff7f85454968d exported/http_objects/image.html
0eaa429c2479a99f839549c87027f90db2e78bfe2dd1117c093903b177ea45b7 exported/http_objects/lab_photo.png

(root@1f1f5ef2d179)~/SBT-DF203-Lab2
#
```

Step 13 Does curl Fetch Images Too? Testing curl Against a Real Browser

The last part of this lab was to test something specific: does a simple tool like curl behave the same way a real browser does when it downloads a page that has an image embedded in it? My guess going in was no and that turned out to be correct, but getting a clean recording of it took a few attempts.

My first two tries failed for timing reasons rather than anything about curl itself: once, I ran the recording as a background job set to stop automatically after 20 seconds, but it actually stopped after about 2 seconds, so curl ran after the recording had already ended and nothing was captured. On the next attempt, I pressed the key that stops a recording before actually running curl in the other window, so again nothing was captured. Both of those are recorded honestly in the screenshots as failed attempts, since documenting what went wrong is part of doing this properly.

On the successful attempt, another detail from Step 6 mattered again: this time I ran curl from inside the container itself, asking for the page at 127.0.0.1 (meaning "this same computer"). Because that request never leaves the container to cross through Docker's network bridge, it doesn't appear on eth0 at all it's genuine loopback traffic, so I had to record on "lo" for this

specific test, the opposite interface from the browser test in Step 6. That's a good example of why it matters to actually understand what a network path is doing, rather than just repeating the same command every time.


```
Terminal Shell Edit View Window Help
sadique — root@1f1f5ef2d179: ~/SBT-DF203-Lab2 — docker exec -it sbtdf203-lab bash — 139x39

(root@1f1f5ef2d179)~/SBT-DF203-Lab2
# tshark -i eth0 -f 'tcp port 80' -a duration:20 -w evidence/curl_only.pcapng &
[1] 155

(root@1f1f5ef2d179)~/SBT-DF203-Lab2
# Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'

# sleep 2

[1]+ Done tshark -i eth0 -f 'tcp port 80' -a duration:20 -w evidence/curl_only.pcapng

(root@1f1f5ef2d179)~/SBT-DF203-Lab2
# curl -v http://127.0.0.1/image.html -o /tmp/image_page.html
* Trying 127.0.0.1:80...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 37408
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
0 0 0 0 0 0 0 0 0 0 0 0 0* using HTTP/1.x
> GET /image.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Fri, 11 Sep 2026 11:26:17 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Thu, 10 Sep 2026 17:22:04 GMT
< ETag: "a1-65b2434e07e37"
< Accept-Ranges: bytes
< Content-Length: 161
< Vary: Accept-Encoding
< Content-Type: text/html
<
{ [161 bytes data]
100 161 100 161 0 0 10231 0
* Connection #0 to host 127.0.0.1:80 left intact
```

```
Terminal Shell Edit View Window Help
sadique — root@1f1f5ef2d179: ~/SBT-DF203-Lab2 — docker exec -it sbtdf203-lab bash — 139x39

# Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'

# sleep 2

[1]+ Done tshark -i eth0 -f 'tcp port 80' -a duration:20 -w evidence/curl_only.pcapng

(root@1f1f5ef2d179)~/SBT-DF203-Lab2
# curl -v http://127.0.0.1/image.html -o /tmp/image_page.html
* Trying 127.0.0.1:80...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 37408
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
0 0 0 0 0 0 0 0 0 0 0 0 0* using HTTP/1.x
> GET /image.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Fri, 11 Sep 2026 11:26:17 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Thu, 10 Sep 2026 17:22:04 GMT
< ETag: "a1-65b2434e07e37"
< Accept-Ranges: bytes
< Content-Length: 161
< Vary: Accept-Encoding
< Content-Type: text/html
<
{ [161 bytes data]
100 161 100 161 0 0 10231 0
* Connection #0 to host 127.0.0.1:80 left intact

(root@1f1f5ef2d179)~/SBT-DF203-Lab2
# wait

(root@1f1f5ef2d179)~/SBT-DF203-Lab2
#
```

```
Terminal Shell Edit View Window Help
sadique -- root@1f1f5ef2d179: ~/SBT-DF203-Lab2 -- docker exec -it sbt

(root@1f1f5ef2d179)~/SBT-DF203-Lab2
# tshark -i lo -f 'tcp port 80' -w evidence/curl_only.pcapng
Running as user "root" and group "root". This could be dangerous.
Capturing on 'Loopback: lo'
10 ^C

(root@1f1f5ef2d179)~/SBT-DF203-Lab2
# tshark -r evidence/curl_only.pcapng | wc -l
10

(root@1f1f5ef2d179)~/SBT-DF203-Lab2
#

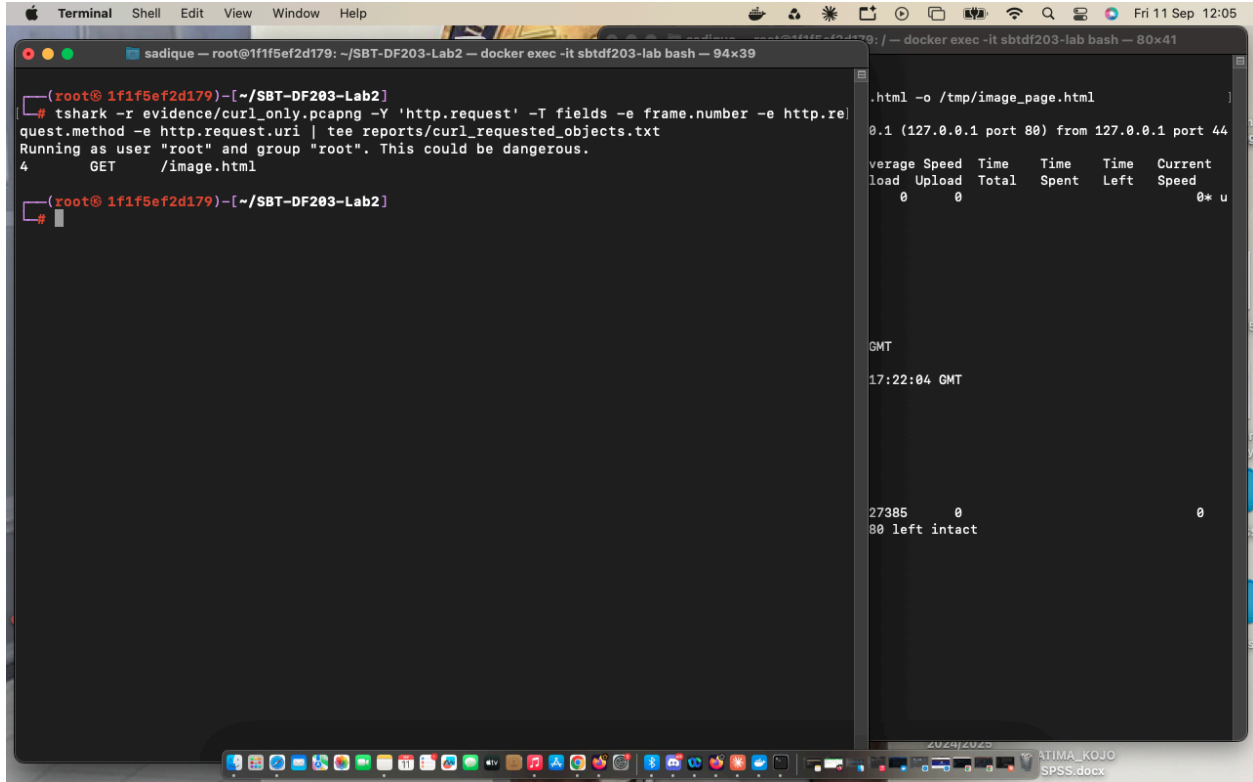
(sadique -- root@1f1f5ef2d179: / -- docker exec -it sbtdf203-lab bash -- 80x41)
(root@1f1f5ef2d179)~/
# curl -v http://127.0.0.1/image.html -o /tmp/image_page.html
* Trying 127.0.0.1:80...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 44714
% Total    % Received % Xferd  Average Speed   Time    Time     Time    Current
           Dload  Upload   Total     Spent    Left     Speed
0         0     0    0    0      0     0      0      0      0
sing HTTP/1.x
> GET /image.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Fri, 11 Sep 2026 12:02:18 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Thu, 10 Sep 2026 17:22:04 GMT
< ETag: "a1-65b2434e07e37"
< Accept-Ranges: bytes
< Content-Length: 161
< Vary: Accept-Encoding
< Content-Type: text/html
<
{ [161 bytes data]
100    161 100    161    0    27385    0      0      0
* Connection #0 to host 127.0.0.1:80 left intact

(root@1f1f5ef2d179)~/
#
```

With that recording running, I asked curl for image.html and let it finish. Pulling the requests back out of that recording showed only one line:

Frame 4 GET /image.html

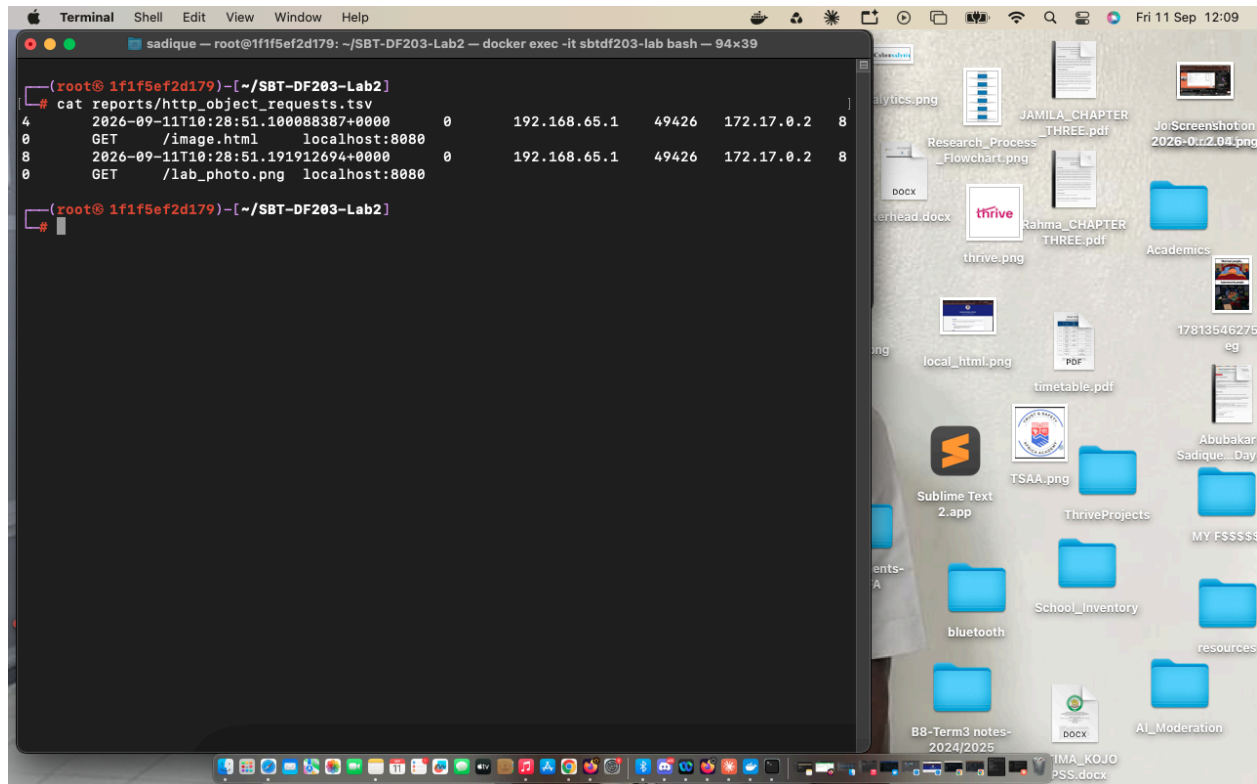
No second request for lab_photo.png ever appeared. curl fetched exactly the one file I told it to, and nothing else.



Putting that side by side with what the browser actually did:

	Requests made	Files fetched
Real browser (Safari)	2	image.html AND lab_photo.png
curl	1	image.html ONLY

The reason for the difference is simple once you see it laid out: a real browser actually reads the HTML it downloads, notices the `<img>` tag inside it, and automatically goes and fetches whatever that tag points to. curl doesn't do any of that it has no idea what's inside the file it just downloaded, it just saves the raw bytes and stops. It only ever fetches exactly what you explicitly ask it for, never anything referenced inside the content itself.



What I Found Summary

Question	Answer
How many things did the browser ask for?	2 the HTML page, then the image
How many things did curl ask for?	1 only the HTML page
What was the image's real size?	151,792 bytes (Content-Length), 152,081 bytes including headers across the segments carrying it
How many pieces was the image broken into?	3 TCP segments, reassembled automatically by Wireshark
Did the recovered files match the originals?	Yes both fingerprinted as byte-for-byte identical
Was the original recording preserved untouched?	Yes original and working copy fingerprints matched exactly
Why did the browser's traffic show on eth0 instead of lo?	It arrived through Docker's port-forward from outside the container, not from inside it
Why did curl's traffic need lo instead of eth0?	curl ran inside the container itself, talking to 127.0.0.1 that never leaves the container

Lessons Learnt

The biggest thing this lab made clear is that a webpage is rarely just "one thing." The moment a page references an image, a browser quietly turns that one page load into multiple separate network requests, each one fetchable, reassemblable, and verifiable entirely on its own. Watching that happen in the actual traffic two distinct requests, two distinct replies, one of them broken into three pieces and stitched back together made something I already knew in theory actually visible and provable.

The curl comparison was just as useful, in a different way. It's easy to assume any tool that can "load a webpage" behaves the same as a browser, but curl doesn't parse or understand the content it downloads at all it just moves bytes. Proving that with a real side-by-side recording, rather than just stating it, is exactly the kind of evidence a forensic analyst would need to back up a claim like that.

Tools and Resources Used

Docker, running a Kali Linux container, as the isolated environment for the whole lab. Apache, curl, tshark, and ImageMagick's identify command, all running inside that container. Git and GitHub, to store and submit the evidence, reports, and screenshots. Google, for looking up a couple of unfamiliar error messages along the way. Claude AI, used strictly as a guide for commands, explanations, and organization. Every command was run by me, on my own machine, and every piece of data in this report came from my own capture.

Appendix: Commands I Used

Command	What it does, in plain words
tshark -i eth0 -w file.pcapng	Record traffic arriving through the Docker network bridge
tshark -i lo -w file.pcapng	Record traffic that never leaves this computer (loopback)
tshark -r file -Y 'http.request'	Pull out just the requests from a recording
tshark -r file -Y 'http.response'	Pull out just the replies from a recording
tshark -r file --export-objects http,DIR	Recover the actual files (HTML, image) out of a recording
tshark -r file -T fields -e tcp.segment.count	Show how many pieces a large reply was split into
identify file.png	Confirm a file is a genuine, valid image and show its dimensions
sha256sum file	Make a digital fingerprint of a file, to prove it hasn't changed

All 20 numbered screenshots referenced above are included in the screenshots folder of the submitted project, in the order the steps were performed.

Student Declaration

I confirm that I did this lab myself, in a safe and isolated setup, kept the original recording untouched, never touched anyone else's computer or network, and that everything written above including the failed attempts is my own work and my own honest record of what happened.

Signed: Abubakari-Sadique Hamidu

Student ID: 2025-FWSD-11392

Date: 10 September 2026